

OOPS

1. **Runtime Polymorphism** - This is also known as **dynamic polymorphism**. The decision about which method to execute is delayed until the program is actually running. This is achieved through **method overriding**, where a subclass provides its own implementation of a method that it inherited from a parent class.

2. **Compile Time Polymorphism** - This is also known as **static polymorphism**. The decision about which method to call is made by the compiler when you compile your code. It's achieved through **method overloading**, which means having multiple methods in the same class with the **same name** but **different parameters**. The difference can be in the number of parameters or the type of parameters.

The Stack: This is a highly organised and efficient region of memory. It stores **primitive types** (like `int`, `char`, `boolean`) and **references** to objects. Every time you call a method, a new "frame" is pushed onto the stack to hold its local variables. When the method finishes, its frame is popped off. It's fast and automatically managed.

The Heap: This is a large, less-organised pool of memory where all the **actual objects** are stored. When you use the `new` keyword, you are allocating memory in the heap for a new object. This area is managed by the Garbage Collector.

You can always assign a child object to a parent reference, but you cannot assign a parent object to a child reference.

```
Shape myShape = new Rectangle( );  
myShape = new Circle( );
```

`myShape` is the reference, `Rectangle` is the object. **Upcasting**.

Since it is procedural programming, in runtime, this does not give error, it just changes what it is referencing to.

Reference type is `Shape`, this decides what methods I can call, so I can only call methods in `Shape`, not the ones in `Rectangle` or `Circle`.

Object type is `Rectangle/Circle` respectively, this decides what version of method (override) is used. This is decided at runtime.

If I have a method in `shape` and an overridden method in `rectangle`, and I call that method, it will call the `rectangle` one. The main thing is that the method must be also there in `shape`.

```
Rectangle myShape = new Shape( ); —>gives COMPILE TIME error
```

Explicit down-casting to deal with this.

```
Shape myShape = new Rectangle( );  
Rectangle myRect = (Rectangle) myShape;
```

Compile Time Error vs Run Time Error

Think of compile time errors being due to syntactic errors, errors in individual codes in ur line, errors in type casting or calling methods that don't exist etc. Changing value of a final variable gives compile time error.

Run Time errors are when something procedurally wont make sense, such as memory allocation.

Something like -

```
int[ ] numbers = {1,2,3};  
System.out.println(numbers[3]);
```

This would give a runtime error, jvm knows size of array only at runtime. Java allocates memory for array at runtime.

Mostly, heap is dealt with in runtime and since arrays are an object, they are stored in a heap.

```
final int[] arr = {1, 2, 3};  
arr[1] = 10;  
System.out.println(arr[1]);
```

This does not give error, final makes the reference variable arr immutable, that it cannot point to a new array. The values of array can be changed. Thus output is 10.

When two same objects are created and referenced separately , jvm makes the first object, and for the second one, it checks if the object already exists, if yes then it points to it.

```
String s1 = "hello"; —>creates an object and references it  
String s2 = "hello"; —>since object already exists, s2 also references it. (Concept of String Pool)  
String s3 = new String("hello"); —> creates a new object and references it
```

Thus, s1 == s2 is true (i.e same reference) and s1.equals(s2) is true i.e same content.
However, s1 == s3 is false, s1.equals(s3) is true.
String s4 = s3.intern();

This looks for a version of “hello” [from string s3] in the pool, if it exists, returns the reference of that i.e the address, thus stored in s4, so s1 s2 and s4 are pointing at the same object in heap. If it does not exist in the pool, it creates the object and references that.

If I have

```
Package p = new ExpressPackage(10, 15);  
p.package_method — works  
p.express_package_method —> compile time error unless its also in package class and  
overridden in express package
```

We cannot instantiate an interface, but we can still create an object of some other class type (i.e instantiate it by some class) and have its reference type be the interface, in this case we can only call methods mentioned in the interface, and their overridden methods in the class would be executed.

```
Dog extends Animal
Animal a = new animal();
Dog d = (dog) a;
```

Compiler will allow casting as long as within same hierarchy. Since the reference type `Animal` can be a `dog` (hierarchy) compiler doesn't say anything. But this will give **RUNTIME ERROR (`ClassCastException`)** since object type is `animal`, the reference cannot be downcasted from `animal` to `dog`. If it was `Animal a = new dog();` then type casting would work. However if instead of casting, like in the code above, you wrote

`Dog a = new animal();` then **COMPILE TIME ERROR**.

Static nested class -> if calling inside an instance(non static) method (example inside a method of the outer class) or in a static method (the main method, outside the class), then

```
Inner inner = new Inner();
Outer.Inner inner = new Outer.Inner();
```

BOTH WORK.

But for non static inner class, must be called with the object.

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
```

OR[in first case in LHS, `Outer.Inner` isn't necessary, just `Inner` works] -> **VERIFY**

```
Outer.Inner inner = new Outer().new Inner(); //compact form
```

This is the case for when accessing in a static method/outside the class. But let's say inside the class, in an instance method, then :-

```
Inner inner = new Inner();
```

This works **ONLY** because Java automatically uses "this" keyword. Better to write as follows-

```
Inner inner = this.new Inner();
```

LOCAL INNER CLASSES (i.e declared inside a method) cannot have any access modifier. Cannot have static term either. They can access local variables of the method only if they are `final` / effectively `final`. Otherwise compile time error.

If you are chaining constructors by `this()` to call other (overloaded) constructors, or when calling superclass constructors using `super()` inside a constructor then it must be the first line of the body else gives compilation error.

SHADOWING -

Q4. Given the following shadowing scenario, how do you access the `x` of the `Outer` class from inside the `Inner` class?

Java

```
class Outer {
    int x = 10;
    class Inner {
        int x = 20;
        void print() {
            int x = 30;
            // INSERT CODE TO PRINT 10
        }
    }
}
```

- A. `System.out.println(super.x);` B. `System.out.println(this.x);` C. `System.out.println(Outer.x);` D. `System.out.println(Outer.this.x);`

Answer: D

Detailed Nuance:

- **Local `x` (30):** Accessed simply as `x`.
- **Inner `x` (20):** Accessed as `this.x` (referring to the Inner instance).
- **Outer `x` (10):** This requires the special syntax `OuterClassName.this.variable`.
- **Why C is wrong:** `Outer.x` implies `x` is a `static` variable of `Outer`. Since `x` is an instance variable (10), you must access it via the specific instance `Outer.this`.

Order of Execution : Parent Constructor -> Initialiser blocks and fields in order of appearance -> Child constructor

Static methods cannot be overridden, they are hidden instead. METHOD HIDING.
Methods cannot be overloaded simply by changing only the return type, gives compilation error.

The only difference is that Abstract classes cannot be instantiated. They can have 0 abstract methods, its not necessary to have an abstract method.

Interfaces can EXTEND other interfaces, not IMPLEMENT other interfaces
Only static nested classes can have static members

```
Java 

// Package 1
package p1;
public class Base {
    protected int x = 10;
}

// Package 2
package p2;
import p1.Base;

public class Derived extends Base {

    public void test() {
        // ✅ ALLOWED: Accessing 'x' through inheritance (your own copy)
        System.out.println(this.x);
        System.out.println(super.x);

        // ❌ ERROR: Accessing 'x' via a generic reference
        Base b = new Base();
        // System.out.println(b.x); // COMPILER ERROR!
    }
}
```

Parent class loaded -> parent static block -> child class loaded -> child static block
then parent object created -> parent initialiser block and fields -> parent constructor
-> child object created -> child init block and field -> child constructor -> any child
methods